

Lecture 8 Execution Control with For Loops and Introduction to Graphing

- Loops are a fundamental construct of execution control.
- Loops are a way of controlling repetition of code, i.e., allow the same piece of code to run multiple times.

I am going to introduce in a small way, something new here, which is making graphs using the `pyplot` submodule of `matplotlib`. We will talk about it more next week as well.

In [1]:

```
import numpy as np
from numpy import random
#Lets start the random number generator
rng = random.default_rng(seed = 9876)
#this is a new module that makes graphs.
from matplotlib import pyplot as plt
#this is a different syntax. i am loading a specific submodule and giving it a new name, plt
```

for and while loop

- Two repetition structures in Python are `for` loops and `while` loops.
- `for` loops run a set number of times. In a typical application, a block of code needs to be applied to the elements of a list or an array or list one at a time. *The number of repetitions of the code is determined in advance.*
- `while` loops run as long as a specific logical condition is true. The key difference between `while` and `for` loops is that *the number of repetitions in a while loop is unknown*

In [2]:

```
#Example 1  
i = 0 #since i is the variable tested in the logical statement, it must be set to an initial value  
while i<4: # this is the conditional statement which controls execution  
    print(i)  
    i = i+1 #this is a critical line, as it updates the value of i
```

```
0  
1  
2  
3
```

- To express the same operations as a `for` loop I need to think about the possible set of values I want `i` to take.
- Then, I want to ask python to repeat the `print` statement for each value of `i`

In [3]:

```
values = [0,1,2,3]  #these are the numbers I want to print. I made a list here.  
for i in values:  
    print(i)
```

0
1
2
3

- The output is identical.
- The critical difference is that I know exactly what values i want the variable i to take in a for loop.
- In a 'while' loop, I use a **logical statement** on the variable i to determine how many times to run it.

For Loop

- A `for` loop is a repetition structure where a code block runs a specified number of times.

```
for var in the_range_of_var:  
    do this  
    and this  
    and this
```

- **the_range_of_var** can be given in many forms. The most common way is the `range` function, but it can also be a **list** or a numpy **array** as we will discuss further below.
- The most important property of a `for` loop is that a `for` loop *automatically increments the value of **var** to the next element in the_range_of_var* in each repetition of the code block
- As we will see in the examples below **var** is often used as an index variable that indexes into arrays sequentially.

The range Function

- creates an integer list spanning a *range*. Very useful for constructing for loops.

In [4]:

```
for i in range(4):  
    print(i)
```

```
0  
1  
2  
3
```

- Python's `range` function can be customized by supplying up to three arguments. The general format of the range function is below:
`range(start,stop,step)`
- *start, stop, step* must be **integers**

In [5]:

```
#here range(start,stop,step) - start is 0 (inclusive) stop is 6 (exclsuive) and 2 is the step size.  
for i in range(0,6,2):  
    print(i)
```

```
0  
2  
4
```

- range can even count down if necessary

In [6]:

```
for i in range(5, -6, -1):  
    print(i)
```

```
5  
4  
3  
2  
1  
0  
-1  
-2  
-3  
-4  
-5
```


- To examine the contents of `range`, convert to a list or a numpy array before printing.
- Once you get comfortable using `range`, you won't need to do this.

In [7]:

```
x = list(range(0,6,2))  
print(x)  
y = np.array(range(0,6,2))  
print(y)
```

```
[0, 2, 4]  
[0 2 4]
```

Loops over lists and arrays

- `for` loops can take a list or array for the variable to iterate over in the loop.

In [8]:

```
mylist = ['I ', 'wish ', 'Kawhi ', 'had ', 'better ', 'knees']  
# the other way to do it is to use range over the length of the list (4):  
for i in mylist:  
    print(i)
```

I
wish
Kawhi
had
better
knees

- Iterating over a list or array usually is done by using an index variable.
- Even in python, its important to understand how this works because you often need to iterate over multiple arrays at the same time.
- To do this, instead of the `for` loop iterating over a variable, it will iterate over the *index* into the array.

In [9]:

```
mylist = ['I ', 'wish ', 'Kawhi ', 'had ', 'better ', 'knees']  
nlist = len(mylist) #I know its 4, but its good to automatically get this.  
for i in range(nlist): # I use range(nlist) to make i go over the values 0,1,2,3 which are index into mylist.  
    print(mylist[i]) #here I index into mylist and then print.
```

```
I  
wish  
Kawhi  
had  
better  
knees
```

- Why is this way of thinking about it useful. Here is a simple example.

In [10]:

```
names = ['a','b','c','d']
grades = [95,82,90,88]
nstudents = len(names)
# the len function gets you the length of the list names
for istudent in range(nstudents):
    # istudent is my indexing variable
    print(names[istudent], ' : ', grades[istudent])
    # i use istudent as an index into both names and grades.
```

```
a : 95
b : 82
c : 90
d : 88
```

- enumerate is a useful python function for making for loops over a list

In []:

```
for index, item in enumerate(names):  
    print(item)  
    print(index)
```

```
a  
0  
b  
1  
c  
2  
d  
3
```

In [12]:

```
#when using enumerate, you keep track of two things, the index into the list, and the item in the list:  
for index, item in enumerate(names):  
    print(item, ' : ', grades[index])
```

```
a : 95  
b : 82  
c : 90  
d : 88
```

Indexing Loops

- The most useful way to think about `for` loops is like this,

```
for index in Variable_to_iterate_over
```

- `index` is the key to the `for` loop. Inside the `for` loop `index` allows you to
1. iterate over the elements of a list or array that must be operated on.

```
y = x[index]**2
```

1. systematically increase the value of a variable,

```
y = 2*index+1
```

2. iterate over multiple lists or arrays in parallel.

```
z = y[index] + x[index]
```

3. control indexing into an output array.

```
y[index] = x[index]**2
```

Graphing

- I am going to introduce here how to make simple bar graphs and line graphs.
- Later in the course, we will do more advanced graphical representations.
- But, its useful to be able to quickly make a graph to look at some numbers, instead of always printing out.

`pyplot` submodule

- The vast majority of the data visualization make use of a single submodule called `pyplot`
- In today's lesson I am gong to introduce two functions
 - `bar` to make a bar graph
 - `plot` to make a line or symbols graph

Organizing input and output arrays with a for loop.

- These are not the best use cases, but I chose these, because they should feel familiar.
- Example: Projecting your bank account after 10 years
- Suppose you have \$500 in your bank account and interest rate is 6.0% compounded annually.
Write a for loop that will compute how much you will have in **nyears**.

In [13]:

```
nyears = 10
# number of years
interest = 0.06 # interest rate
balance = 500 # starting balance
for year in range(1,nyears+1): #notice I do nyears+1 here. This is because year 0 is the first value and then
                                # I want to interate nyears more years.
    balance = balance+interest*balance
print('Balance of ', balance, 'after ', nyears, 'years')
```

Balance of 895.4238482714267 after 10 years

- If I want to keep track of the balance after each year, I could use

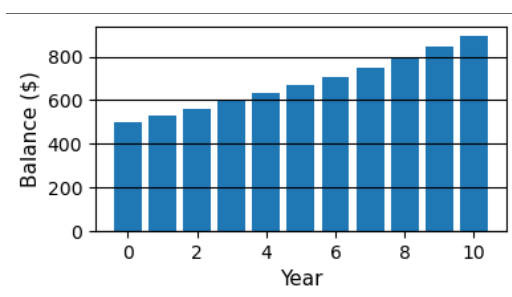
In [14]:

```
nyears = 10# number of years
balance = list() #I make an empty list to hold my answers
interest = 0.06 # interest rate
balance.append(500) # starting balance at year 0
years = [0] #I make a list to track the years starting at 0
for year in range(1,nyears+1): #again, notice I use range with nyears and my index variables is years,
    newbalance = balance[year-1]+interest*balance[year-1]
    # notice I used the index year -1.
    #to get year n, I have to calculate using the balance from n-1
    balance.append(newbalance) #append the new value of newbalance to the list.
    years.append(year) #append the value of year to years
print('balance = ', balance)
print('years = ', years)
```

```
balance = [500, 530.0, 561.8, 595.5079999999999, 631.2384799999999, 669.1127887999999, 709.25955
61279999, 751.8151294956799, 796.9240372654207, 844.739479501346, 895.4238482714267]
years = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [15]:

```
### Let's make a nice bar plot of it.  
plt.figure(figsize = (4,2)) #this makes an empty canvas.  figsize = (width, height)  
plt.bar(years,balance) #this makes a bar plot  
#notice I control the x variable as years and y variable as balance  
plt.xlabel('Year',fontsize = 11) # i label my x axis  
plt.ylabel('Balance ($)',fontsize=11) # i label my y axis  
plt.grid('on',axis='y',color='k') # I make grid lines to increase readability, but only on the y-axis  
plt.show() #i need this to make the plot look pretty.
```



- What if interest rate varies over the next 10 years? So that the interest rate (percentage) for each of the five years is

```
interest = [6,8,4,4,8,6,7,5,5,7]
```

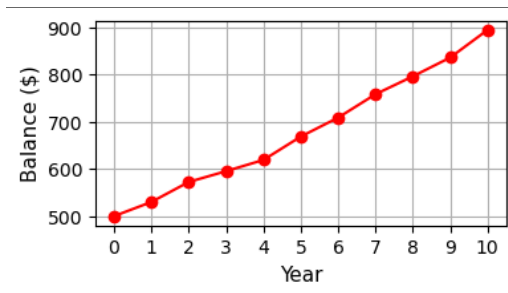
In [16]:

```
interest = [0.06,0.08,0.04,0.04,0.08,0.06,0.07,0.05,0.05,0.07]# interest rate in decimal
nyears = len(interest) # I determined the number of years from the lenght of interest using the size function
balance = list() #empty list to hold the balance of each year
balance.append(500) # starting balance
years = [0] #I started the years list at 0.
for year in range(1,nyears+1):
    newbalance = balance[year-1]+interest[year-1]*balance[year-1]
    #notice that I use year-1 here as an index into interest
    #and into balance
    #I use year -1 to get the previous years data.
    balance.append(newbalance) #i append the newbalance to the list balance
    years.append(year) #I append the years to the list years
print('balance = ', balance)
print('years = ', years)
```

```
balance = [500, 530.0, 572.4, 595.2959999999999, 619.1078399999999, 668.6364671999999, 708.75465
52319999, 758.3674810982399, 796.2858551531518, 836.1001479108095, 894.6271582645661]
years = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [17]:

```
### Let's make a line graph
plt.figure(figsize = (4,2)) # I define my canvas size
plt.plot(years,balance,'ro-') # notice I control the x variable as years and y variable as balance
                                # 'ro-' refers to the color of the line 'r' = red, the marker 'o' circle and '-' is a line
plt.xlabel('Year',fontsize = 11) #label the x-axis
plt.xticks(years) # I want a tick mark for every year for readability
plt.ylabel('Balance ($)',fontsize=11) #y-axis label
plt.grid('on') # now i let grid default to putting grid lines on x and y
plt.show()
```



- Lets consider doing this problem with arrays instead of lists.
- Lets go back to simple case of a single interest rate first.
- when working with arrays, we have to preallocate the array

In [18]:

```

nyears = 10# number of years
#when working with arrays, we can preallocate the array with zeros
balance = np.zeros(nyears+1)
#I need 1 more year because I start at year 0
interest = 0.06 # interest rate
balance[0] = 500 # starting balance at year 0 - notice i use array indexing
years = np.arange(nyears+1) #I can use np.arange to fill out the years.
for year in range(1,nyears+1): #again, notice I use range with nyears+1, starting at 1
    # and my index variables is year,
    balance[year] = balance[year-1]+interest*balance[year-1]
    # notice I am using indexing in BOTH the output and input side of the calculation.
print('balance = ', balance)
print('years = ', years)

```

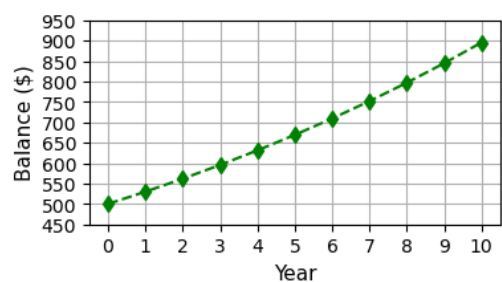
```

balance = [500.          530.          561.8         595.508        631.23848
669.1127888  709.25955613  751.8151295  796.92403727  844.7394795
895.42384827]
years = [ 0  1  2  3  4  5  6  7  8  9 10]

```

In [19]:

```
### Let's make a graph again
### Let's make a line graph
plt.figure(figsize = (4,2)) # I define my canvas size
plt.plot(years,balance,'gd--') # notice I control the x axis variable here using np.arange to go from 0 to 11
                                # 'gd--' refers to the color of the line 'd' = red, the marker 'd' diamonds
                                # and '--' is a dashed line
plt.xlabel('Year',fontsize = 11) #label the x-axis
plt.xticks(years) # I want a tick mark for every year for readability
plt.yticks(np.arange(450,1000,50)) # I decided I want y ticks every 50
plt.ylabel('Balance ($)',fontsize=11) #y-axis label
plt.grid('on') # now i let grid default to putting grid lines on x and y
plt.show()
```



In []: